Recurrent Neural Network

CSC3034 Computational Intelligence

Dr Tang Tiong Yew

Lecture outline

ANN with Recurrent Architecture

Recurrent Neural Networks (RNN)

Long Short-Term Memory (LSTM)

Gaterd Recurrent Units (GRU)

Comparisons

Consideration in Neural Network Design

ANN with Recurrent Architecture

Recurrent Neural Networks (RNNs), **Long Short-Term Memory** (LSTM) networks, and **Gated Recurrent Units** (GRU) are among the most prominent architectures particularly in tasks involving sequences of data like

- time series forecasting,
- natural language processing (NLP), and
- video analysis.

All of them include feedback mechanisms to incorporate sequential information.

Recurrent Neural Networks (RNN)

Recurrent Neural Networks (RNNs) are a class of neural networks designed for sequential data.

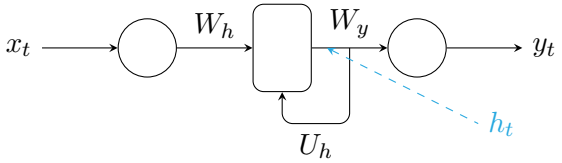
RNNs have feedback loops that allow information to persist, making them capable of remembering previous inputs and using them to influence future predictions.

Recurrent Neural Networks (RNN)

Architecture

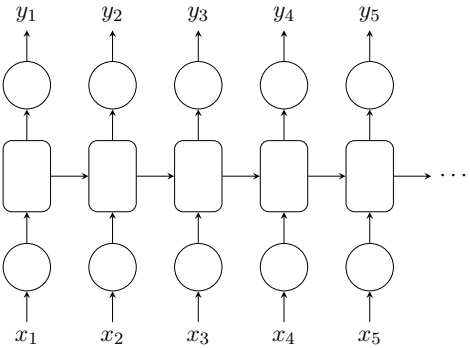
The RNN structure has a repeating loop (recurrent) that passes hidden states from one time step to the next. At each time step, an RNN takes in two inputs:

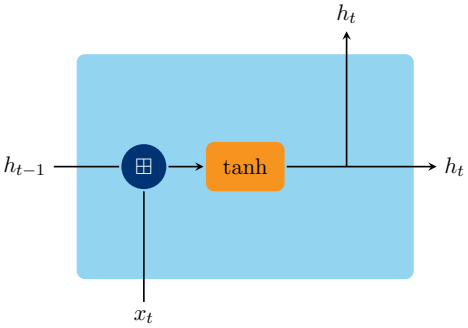
- Current input x_t: The input at the current time step
- Previous hidden state h_{t-1}: The hidden state from the previous time step



Recurrent Neural Networks (RNN)

Architecture





The hidden state is updated based on these two inputs

$$h_t = \tanh(W_h \cdot x_t + U_h \cdot h_{t-1})$$

Training

Backpropagation Through Time (BPTT) is used to train RNNs. Similar as error backpropagation, where errors are propagated backwards.

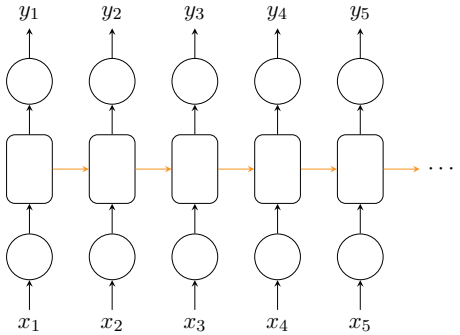
The network is unrolled across time steps, and the gradients are calculated across the sequence.

Strength

RNNs have short-term memory and are ideal for handling sequences where the relationship between the current and previous steps is crucial.

Vanishing/Exploding Gradient Problem

During backpropagation, the gradients either shrink (vanish) or grow too large (explode) as they are passed back through many time steps. This limits the network's ability to learn long-term dependencies.



Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) networks are a specialized type of RNN designed to solve the vanishing gradient problem and to capture long-term dependencies more effectively.

Long Short-Term Memory (LSTM)

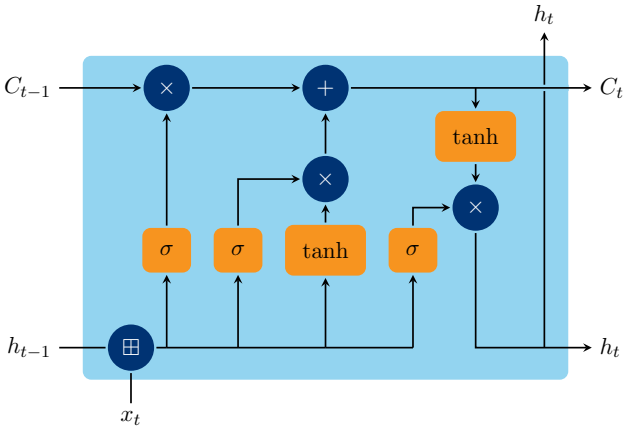
Architecture

The core of LSTM is its memory cell, which can retain information for long periods. It also has gates to control the flow of information:

- **Forget Gate**
  Decides what information from the cell state should be discarded.
- **Input Gate**
  Decides which information should be updated in the cell state.
- **Output Gate**
  Decides what information from the current cell should be passed to the next hidden state.

Long Short-Term Memory (LSTM)

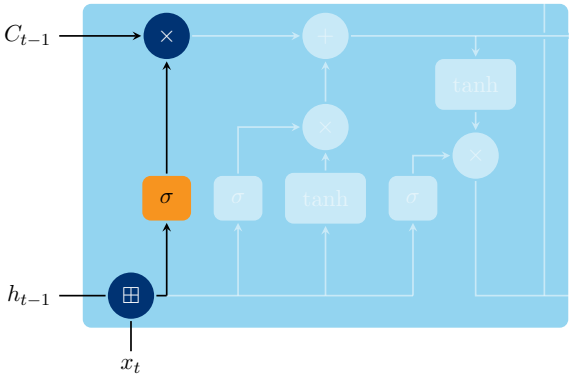
Architecture



- Forget Gate
- Input Gate
- Output Gate

Long Short-Term Memory (LSTM)

Architecture



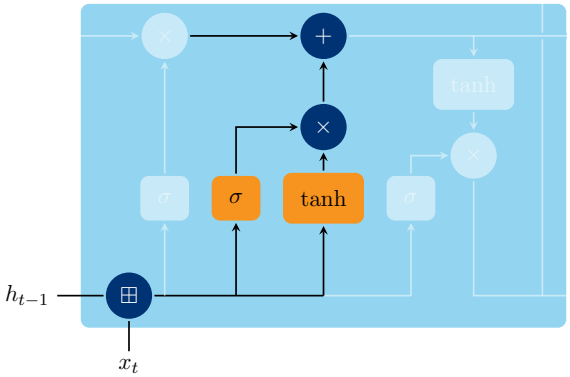
Forget Gate

Decides what information from the cell state should be discarded.

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right)$$

Long Short-Term Memory (LSTM)

Architecture



Input Gate

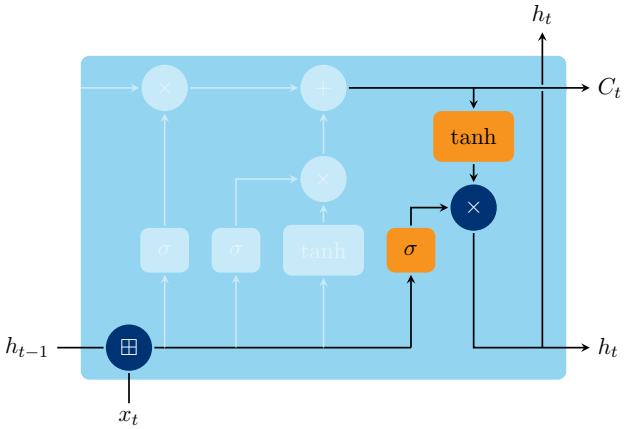
Decides which information should be updated in the cell state.

$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] + b_i\right)$$

$$\tilde{C}_t = \tanh\left(W_C \cdot [h_{t-1}, x_t] + b_C\right)$$

Long Short-Term Memory (LSTM)

Architecture



Output Gate

Decides what information from the current cell should be passed to the next hidden state.

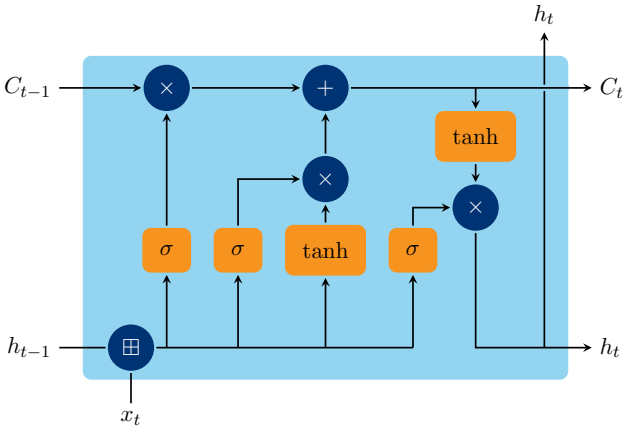
$$o_t = \sigma\left(W_o \cdot [h_{t-1}, x_t] + b_o\right)$$

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

$$h_t = o_t \cdot \tanh(C_t)$$

Long Short-Term Memory (LSTM)

Architecture



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right)$$
$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] + b_i\right)$$
$$o_t = \sigma\left(W_o \cdot [h_{t-1}, x_t] + b_o\right)$$
$$\tilde{C}_t = \tanh\left(W_C \cdot [h_{t-1}, x_t] + b_C\right)$$
$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$
$$h_t = o_t \cdot \tanh(C_t)$$

Long Short-Term Memory (LSTM)

Strengths and Limitations

- LSTMs can remember information across many time steps, which makes them more effective than RNNs in learning long-term dependencies.
- The gates allow the network to explicitly control what to remember, forget, or output.
- Solves the vanishing gradient problem better than standard RNNs.
- Effective for long sequences where important information may be distributed far apart in time, such as in speech recognition, text generation, and time series prediction.

Gaterd Recurrent Units (GRU)

Gated Recurrent Units (GRU) are a variation of LSTMs that aim to simplify the model while maintaining similar performance.

GRUs combine the forget and input gates into a single gate called an update gate, and they eliminate the output gate, simplifying the overall architecture.

Gaterd Recurrent Units (GRU)

Architecture

GRU has two main gates

- **Reset Gate** r_t

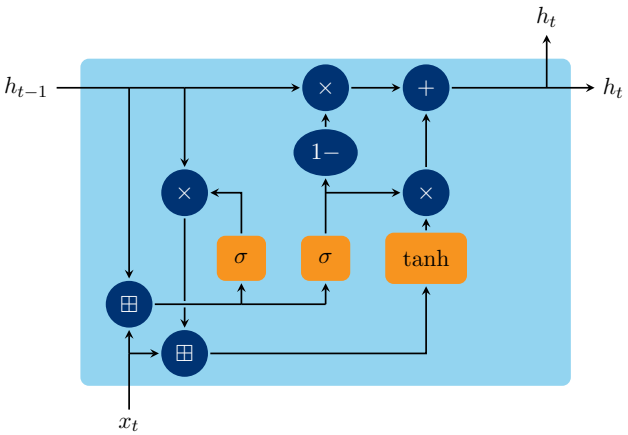
  Determines how much of the previous hidden state to forget before combining it with the new input.

- **Update Gate** z_t

  Decides how much of the previous hidden state to retain and how much of the new input to incorporate.

Gaterd Recurrent Units (GRU)

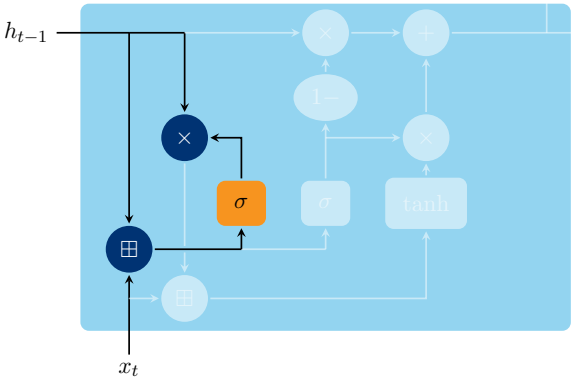
Architecture



- Reset Gate
- Update Gate

Gaterd Recurrent Units (GRU)

Architecture



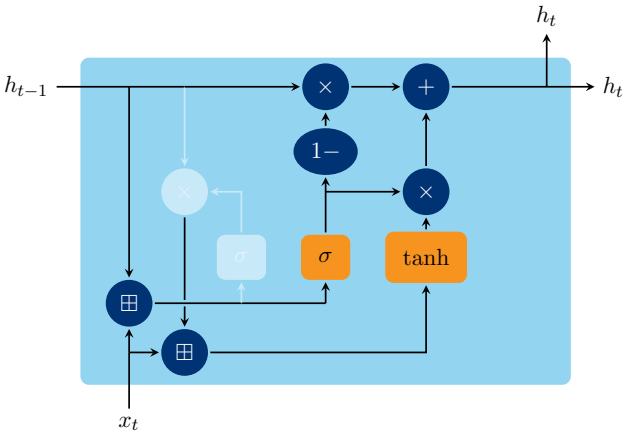
Reset Gate

Determines how much of the previous hidden state to forget before combining it with the new input.

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t] + b_r\right)$$

Gaterd Recurrent Units (GRU)

Architecture



Update Gate

Decides how much of the previous hidden state to retain and how much of the new input to incorporate.

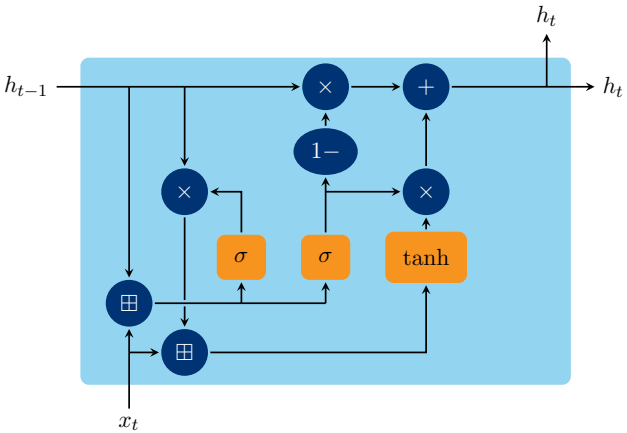
$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t] + b_z\right)$$

$$\tilde{h}_t = \tanh\left(W_h \cdot [r_t \cdot h_{t-1}, x_t] + b_h\right)$$

$$h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot \tilde{h}_t$$

Gaterd Recurrent Units (GRU)

Architecture



$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t] + b_r\right)$$
$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t] + b_z\right)$$
$$\tilde{h}_t = \tanh\left(W_h \cdot [r_t \cdot h_{t-1}, x_t] + b_h\right)$$
$$h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot \tilde{h}_t$$

Gaterd Recurrent Units (GRU)

Strengths and Limitations

- GRUs are simpler than LSTMs because they have fewer gates.
- They tend to train faster and may work as well as LSTMs on certain tasks.
- Fewer parameters compared to LSTM (since fewer gates), leading to a simpler model.
- Often faster to train due to fewer computations per time step.
- Provides similar performance to LSTMs for many tasks, especially with medium-length sequences.

Comparisons

Memory handling

RNN	GRU	LSTM
Struggles with long-term memory due to vanishing/exploding gradient issues.	Handles long-term dependencies similarly to LSTMs but does so with fewer gates and a simpler architecture.	Strong ability to retain long-term dependencies due to its memory cell and gating mechanism.

Comparisons

Complexity

RNN	GRU	LSTM
Simplest architecture, but prone to performance degradation over long sequences.	Slightly simpler than LSTM with two gates (update and reset), leading to faster training and comparable performance in many cases.	More complex, with three gates and a memory cell, which allows it to perform better in tasks requiring memory over long time spans.

Comparisons

Training Time

LSTM	GRU	RNN
Slower to train due to its complexity but more robust for long sequences.	Faster to train than LSTM while maintaining similar performance.	Generally faster to train than LSTM and GRU but less effective in terms of learning longer-term dependencies.

Comparisons

Use Cases

RNN	GRU	LSTM
Short sequence tasks like basic sentiment analysis or where past information is less critical.	Medium-length sequences where computational efficiency is needed without sacrificing too much accuracy, e.g., real-time predictions or chatbots.	Long sequence tasks like machine translation, speech-to-text, handwriting recognition, and music generation.

Design a neural network

- Data conditioning
- Number of hidden layers
- Number of neurons in hidden layers
- Activation functions
- Connections
- Learning algorithm

Design a neural network

Data conditioning

- Representation of input data (categorical, continuous, discrete).
- Data for training, testing, and validation.
  - the training data is used to train the neural network,
  - from time to time, the performance of the neural network is evaluated using the validation data,
  - when the neural network training is terminated, the neural network is tested with the testing data.
  - training, validation, and testing data are independent of each other.
  - the available data is split into different proportions.
  - training data has the largest proportion; proportions for validation and testing data are similar.

Design a neural network

Number of hidden layers & number of hidden neurons

- Hidden layers are useful in identifying hidden features in the input patterns.
- Does it mean that we should use as many hidden neurons as possible?
- No. Overfitting might happen.

Overfitting

- The problem of overfitting occurs when the network simply memorise the training data.
- An overfitted neural network is unable to generalise.
- The neural network will be unable to identify the correct output when presented with data that are not part of the training data.
- To prevent overfitting, we should choose the smallest number of hidden neurons that yields good generalisation.
- Occasional check with validation data is another approach to prevent overfitting.

Design a neural network

Activation functions

- As mentioned previously, there are different type of activation functions: linear, sign, step, sigmoid.
- Generally each layer will have the same type of activation function.
- Activation functions in different layers are not necessarily the same type.

Design a neural network

Connections

- Feedforward network
- Recurrent network

Design a neural network

Learning algorithms

- Supervised learning
- Unsupervised learning
  - Hebbian learning
  - Competitive learning

Thank you

CSC3034 Current Course Update

08 Recurrent Neural Network - handout

Synchronized from the current CSC3034 course site on 14 August 2026.

- Current Lab 8b uses a Keras Embedding/LSTM pipeline for IMDb sentiment analysis.
- The practical covers preprocessing, training, evaluation, and prediction.
- Deep-learning requirements are separated from the core lab environment.

Current materials author: Dr Tang Tiong Yew

See the synchronized lab notes and runnable examples for full instructions.